

Robustness Enhancement in Image Classification via Data Augmentation

Submitted by: Jagriti Koirala and Pritesh Das

Datasets Used: CIFAR-10, CIFAR-100, SVHN, CIFAR-10-C

DASC 5304 – Machine Learning | University of Texas at Arlington

Introduction & Why This Matters

The gap

Models perform well on clean test images but fail when inputs are corrupted — blur, noise, JPEG compression. Standard training with random crop and flip does not address this.

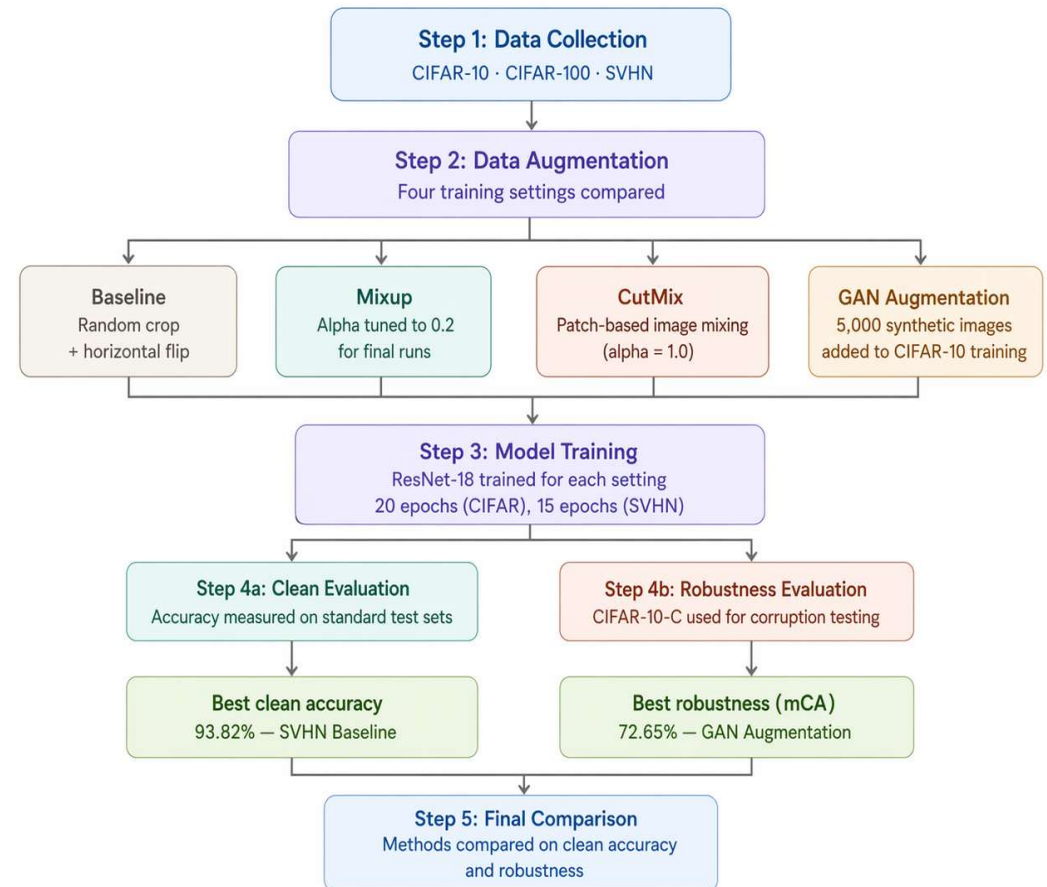
What we set out to test

We trained ResNet-18 under four augmentation strategies and measured both clean test accuracy and mean corruption accuracy (mCA) across 19 corruption types $\times$ 5 severities.

The core question

Does stronger augmentation actually improve corruption robustness, or does it just trade clean accuracy for resilience?

Project Workflow and Evaluation Setup



Datasets & Experimental Setup

Dataset	Classes	Train / Test	Role
CIFAR-10	10	50,000 / 10,000	Primary: clean accuracy + CIFAR-10-C robustness
CIFAR-100	100	50,000 / 10,000	Tests Mixup under greater class pressure
SVHN	10	73,257 / 26,032	Real street digit photos representing natural distribution shift

Training Configuration

Model ResNet-18: final layer adapted for 10 or 100 output classes

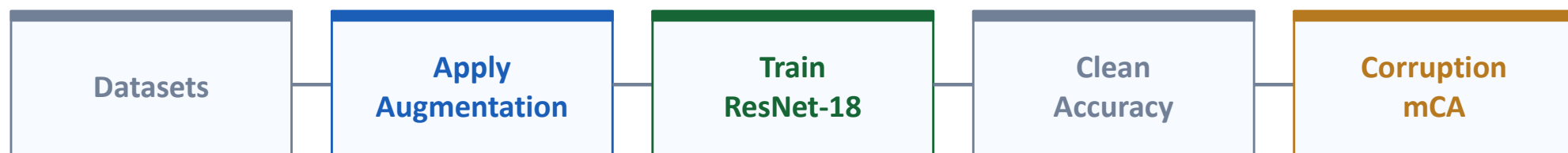
Optimizer Adam, LR = 0.001, Cosine Annealing schedule

Batch / Epochs Batch 128, CIFAR-10 & CIFAR-100: 20 epochs, SVHN: 15 epochs

Baseline augmentation Random crop 32×32 (padding=4) + horizontal flip , which is applied in all methods

Hardware Google Colab, Tesla T4 GPU, PyTorch

How Each Method Works



Baseline	Mixup $\alpha=0.2$	CutMix $\alpha=1.0$	GAN (DCGAN)
Random crop 32×32 (padding=4)	Randomly selects a mixing ratio λ from a Beta distribution ($\alpha = 0.2$)	Rectangular patch from image j	DCGAN trained 50 epochs on CIFAR-10
Horizontal flip only	Blends 2 training images and their labels proportionally by λ	Pasted into image i	5,000 synthetic 32×32 images generated
No label modification	Forces the model to learn smoother decision boundaries between classes	Labels by patch area ratio	Pseudo-labels assigned via baseline

Final Clean Test Accuracy — All Methods & Datasets

MIDWAY CLEAN TEST ACCURACY FOR CIFAR-10, CIFAR-100, AND SVHN.

Dataset	Method	Clean test accuracy (%)
CIFAR-10	Baseline	67.38
CIFAR-10	Mixup ($\alpha = 1.0$)	56.54
CIFAR-100	Baseline	26.38
CIFAR-100	Mixup ($\alpha = 1.0$)	23.10
SVHN	Baseline	84.20
SVHN	Mixup ($\alpha = 1.0$)	83.47

^aMidway results with short training schedules (3–5 epochs).

FINAL CLEAN TEST ACCURACY ON CIFAR-10 FOR DIFFERENT AUGMENTATION SCHEMES.

Method	Clean test accuracy (%)
Baseline	82.94
Mixup ($\alpha = 0.2$)	82.41
CutMix ($\alpha = 1.0$)	79.28
GAN-augmented (baseline + GAN)	82.79

^aAll models use ResNet-18 trained with cosine-annealed learning rates.

FINAL CLEAN TEST ACCURACY ON CIFAR-10, CIFAR-100, AND SVHN.

Dataset	Method	Clean test accuracy (%)
CIFAR-10	Baseline	82.94
CIFAR-10	Mixup ($\alpha = 0.2$)	82.41
CIFAR-100	Baseline	53.83
CIFAR-100	Mixup ($\alpha = 0.2$)	53.34
SVHN	Baseline	93.82
SVHN	Mixup ($\alpha = 0.2$)	93.71

^aAll models are ResNet-18 trained with cosine-annealed learning rates.

- While Analyzing the accuracies of CIFAR-10, CIFAR-100, and SVHN, for the Mix-Up augmentation we tried tuning the alpha value. The different alpha values, we experimented with were 0.2, 0.4, and 1.0. Alpha value at 0.2 gave comparatively the highest test accuracy results, which was why it was chosen for the Mix-Up Augmentation method.

- PP = Percentage (%) Points

Epoch Selection Based on Convergence: Why We Chose 20 Epochs for CIFAR-10/CIFAR-100 and 15 for SVHN ?

- For CIFAR-10, the baseline model achieved its best test accuracy of 82.94% at epoch 19 and finished at 82.85% at epoch 20, while the Mix-Up model reached 82.41% at epoch 20. This shows that performance had already plateaued in the final epochs, so 20 epochs were sufficient to capture convergence.
- CIFAR-100 required a longer run to stabilize, but the final history still showed convergence by the end of 20 epochs. The baseline model reached 53.83% at epoch 19 and slightly decreased to 53.54% at epoch 20, while the Mixup model reached 53.34% at epoch 20, suggesting that further epochs were unlikely to yield large gains.
- SVHN converged faster than CIFAR-10 and CIFAR-100. The baseline model reached 93.82% at epoch 15 and the Mixup model reached 93.71% at epoch 15, showing that high performance was already achieved within a shorter training schedule. The *training_histories_all_models.csv* file uploaded as part of the final submission includes these technical details
- The models converged in the late training stage, where test accuracy stopped increasing meaningfully and only small fluctuations were observed from one epoch to the next. In our setup, this convergence region was around epochs 19 to 20 for CIFAR-10 and CIFAR-100, and around epoch 15 for SVHN.
- Our decision was also motivated by generalization. Once test accuracy stabilizes but training continues, the model would have kept fitting the training data more closely without improving performance on unseen data, so stopping at the observed convergence point helps avoid unnecessary overfitting risk

How Robust Are These Models?

MEAN CORRUPTION ACCURACY (MCA) ON CIFAR-10-C.

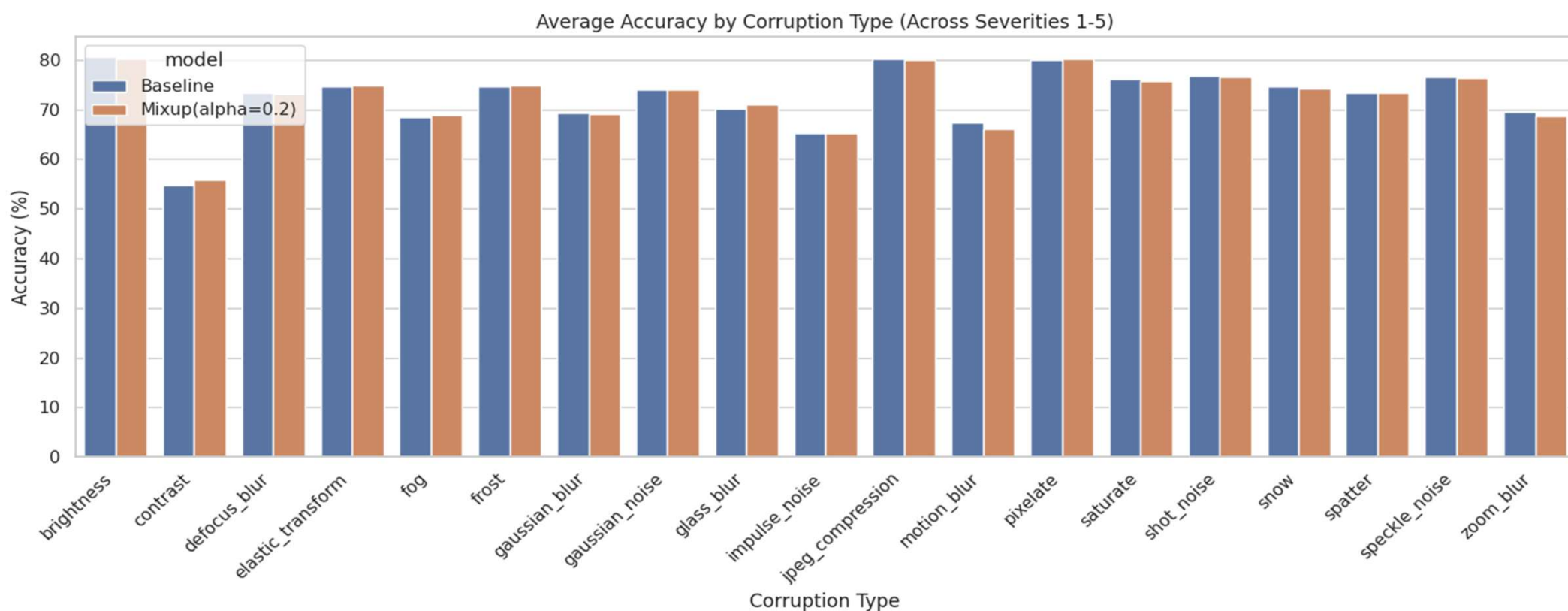
Model	mCA (%)
Baseline	72.573158
CutMix ($\alpha = 1.0$)	67.136000
Mixup ($\alpha = 0.2$)	72.514842
GAN	72.640000

^amCA is averaged over 19 corruption types and 5 severities of CIFAR-10-C

Key Findings

- Baseline and Mixup essentially tied, only 0.06 pp difference (72.57% vs 72.51%). Mixup added no robustness advantage.
- CutMix dropped 5.4 pp below baseline (67.14% vs 72.57%). Patch-based mixing doesn't transfer to pixel-level corruptions.
- GAN edges baseline by just 0.07 pp (72.65% vs 72.57%). Around 5,000 synthetic images produced no meaningful robustness improvement.
- Models with best mCA, in descending order: GAN > Baseline > Mix-Up > Cut-Mix

Per Corruption Breakdown: Baseline vs Mixup



The two models track each other across all 19 corruption types. No corruption clearly favours Mixup. Biggest difference: contrast (+1.1 pp Mixup), motion blur (−1.4 pp Mixup). This confirms the mCA result: Mixup adds no consistent robustness benefit.

Final Phase GAN Augmentation on CIFAR 10

Method:

- Trained a lightweight DCGAN-style generator on CIFAR-10 for 50 epochs.
- Generated 5,000 synthetic 32×32 RGB images from random latent noise and assigned labels using the baseline model as pseudo-labels.
- Retrained ResNet-18 on the augmented dataset consisting of 50,000 real images plus 5,000 synthetic images.

Results:

- Clean accuracy: 82.79% with GAN augmentation vs 82.94% baseline, a drop of 0.15 percentage points (PP).
- mCA robustness: 72.65% with GAN augmentation vs 72.57% baseline, a gain of 0.07 percentage points (PP).

Key takeaways:

- GAN augmentation did not produce a meaningful improvement in the final phase because the clean-accuracy drop and robustness gain were both negligible in magnitude.
- The most plausible reasons are low visual fidelity of generated samples, unconditional image generation, and noise introduced by pseudo-label assignment.

Conclusion: What We Found

1

Baseline was never beaten

Across all 3 datasets and both clean accuracy and mCA, standard ResNet-18 with basic augmentation held its ground against every method we tested.

2

Mixup: marginal gap, no robustness gain

Tuning α from 1.0 to 0.2 recovered the early accuracy drop. Final gap: -0.53 pp clean, -0.06 pp mCA on CIFAR-10. Not a huge gap, but the level of difference as compared to baseline reduced.

3

CutMix hurt both metrics

79.28% clean (-3.7 pp) and 67.14% mCA (-5.4 pp). Patch-based mixing does not generalize to pixel-level corruptions like noise or blur.

4

GAN augmentation: effectively neutral

82.79% clean, 72.65% mCA, essentially tied with baseline. Pseudo-label noise and the 10% of extra data did not add any large benefits.

Final Code Execution Video

Shared with me - Google Drive

Final_Code_ML_Project.ipynb

colab.research.google.com/drive/1Z_P0XmbP5BZjZIMpNhjeH43H0epqxj4d

Ask Gemini

Final_Code_ML_Project.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code +

- Run all (Ctrl+F9)
- Run before (Ctrl+F8)
- Run the focused cell (Ctrl+Enter)
- Run selection (Ctrl+Shift+Enter)
- Run cell and below (Ctrl+F10)
- Interrupt execution (Ctrl+M)
- Restart session (Ctrl+M)
- Restart session and run all
- Disconnect and delete runtime
- Change runtime type
- Manage sessions
- View resources
- View runtime logs
- Deploy to Google Cloud Run

DASC 5304 - Mac

- Project Title:
- Team Member:

Import torch

```
import torch
print(torch.__version__)
```

True Tesla T4

Project Overview

In this project, we will explore the impact of data augmentation techniques on the accuracy and robustness of a deep learning image classifier. We start with a standard baseline model and then introduce various augmentation techniques to see whether they improve generalization performance.

Project Plan

Phase 1 - Train a baseline ResNet18 model on CIFAR-10. Baseline uses standard augmentation (random crop and horizontal flip).

Phase 2 - Add Mixup during training and compare the results.

Phase 3 - Later, test robustness and expand to other datasets.

For this midway submission, we focused on CIFAR-10 first to build and test the full training pipeline before moving to the other datasets.

Phase 1 - Baseline Model Implementation

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
import torchvision.models as models
import matplotlib.pyplot as plt

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)
```

Using device: cuda

Thank You

Jagriti Koirala, Pritesh Das

DASC 5304: Machine Learning, University of Texas at Arlington